SmartBank — Live Demonstration Guide

SmartBank: Database-Driven Fraud Detection and Transaction Monitoring System
Advanced DBMS project · Demonstration guide · Prepared 14 Oct 2026 from the actual repository and a read-only check of the local database

How to read this guide

There are **two different ways** to show SmartBank. Never mix them up while presenting.

	REAL ORACLE MODE (main demonstration)	**DEMO MODE** (backup only)
Where	Your laptop: **http://localhost:5173**	Public link: **https://ashmita-dev.github.io/SmartBank/**
Data	Live Oracle 21c XE database (`SMARTBANK_USER`)	Synthetic seed data, simulated inside the browser
New transfer	Inserted into `BANK_TRANSACTION`, scored by the PL/SQL trigger	Simulated by a JavaScript copy of the rules; **nothing is written to Oracle**
How to recognise it	Green **LIVE** badge (top right)	Amber **DEMO MODE** banner on every page

Labels used below: ✅ **Verified** = checked on 14 Oct 2026. ⚠️ **Confirm** = depends on your installation or was not run, so check it yourself tonight.

1. Project overview

What SmartBank does. SmartBank is a banking database that watches every new transaction and decides whether it looks like fraud. Suspicious transactions get a risk score, a risk level and a fraud alert, and the alert is given to a fraud analyst to investigate.

Why fraud detection is needed. Fraudsters move money quickly, from new devices and unusual places. A bank must spot these patterns the moment a transaction is recorded, not days later.

How Oracle and PL/SQL are used. The fraud logic lives **inside the database**:

1. `TRG_VALIDATE_TRANSACTION` (BEFORE INSERT trigger) rejects inactive accounts and amounts above ₹1,00,00,000.
2. `TRG_FRAUD_MONITOR` (compound trigger) calls `DETECT_FRAUD` for every new row.
3. `CALCULATE_RISK` adds points for four signals; `CLASSIFY_RISK` turns the score into a level.
4. For MEDIUM or HIGH, `GENERATE_ALERT` inserts a `FRAUD_ALERT` and assigns the analyst with the fewest OPEN alerts. HIGH also sets the transaction to **FLAGGED**.
5. Oracle REST Data Services (ORDS) publishes the data as a REST API; a React website shows a **Fraud Analyst** console and a **Customer** portal.

Signal checked by `CALCULATE_RISK`	Points
Amount > ₹1,00,000 / > ₹50,000 / > ₹20,000	+40 / +25 / +10
2 or more / 1 other transaction on the same account in the previous 10 minutes	+25 / +15
Device never used before on this account	+20
Location is not one of the customer's address cities	+20

Score is capped at 100. **LOW 0–39 · MEDIUM 40–69 · HIGH 70–100.** It is a rule-based point total, not a probability.

2. What to open tomorrow

The current state of your laptop (✅ verified 14 Oct 2026, read-only):

- `OracleServiceXE` and `OracleOraDB21Home1TNSListener` are **Running** with startup type **Automatic**: they normally start with Windows.
- ORDS is installed in `C:\ords` with its configuration in `C:\ords-config`, and listens on `127.0.0.1:8080` only.
- API base URL: `http://localhost:8080/ords/smartbank/sb/api`. `GET /kpis` answered with 15 customers, 20 accounts, 102 transactions and 76 fraud alerts.
- The laptop runs the **older REST module (no server-side login)**. This matters for login (section 4).

`scripts\start-backend.ps1` **does all backend work for you.** It starts both Oracle services if they are stopped (asking for administrator permission only if needed). It checks the ORDS pool, starts ORDS in a **new window** (skipping this if port 8080 is already in use), and waits until the API answers. **Do not start ORDS a second time by hand.**

1. Open **VS Code** → **File** → **Open Folder...** → `C:\Projects\SmartBank`.
2. Open a terminal: **Terminal** → **New Terminal** (PowerShell). Rename it **Backend** (right-click the tab → Rename).
3. Open a second terminal (the **+** button) and rename it **Frontend**.
4. Keep **Chrome** ready, and optionally a third terminal named **SQL** for the database proof.

3. Exact startup checklist

#	Terminal	Command / action	Expected result
1	Backend	`powershell -ExecutionPolicy Bypass -File scripts\start-backend.ps1`	Both services `Running`, then green **API is up** and the KPI numbers. If a **UAC** window appears, click **Yes**. A separate **ORDS window** opens: **leave it open** (closing it stops ORDS).
2	Frontend	`cd frontend`	—
3	Frontend	`npm run dev`	`Local: http://localhost:5173/`
4	Chrome	open **http://localhost:5173**	SmartBank login page **without** an amber DEMO MODE box
5	Chrome	sign in as analyst (section 4)	Green **LIVE** badge; sidebar says **API online**
6	Chrome	**System & Settings → Test all endpoints**	**10 healthy · 1 failed of 11.** `GET /health` fails because the older module has no health endpoint, which is expected. Every other row should be HTTP 200.

⚠️ **Confirm tonight:** steps 1–6 on your own laptop. `npm run dev` needs the dependencies already in `frontend\node_modules`; if it says *vite is not recognized*, run `npm ci` once inside `frontend` (internet needed).

Optional check in a browser tab: `http://localhost:8080/ords/smartbank/sb/api/kpis` should show JSON with `total_customers: 15`.

If the local Oracle environment fails (and section 9 does not fix it within 5 minutes), open the public link **https://ashmita-dev.github.io/SmartBank/** and say clearly: "*This is the public demo mode. It runs a simulation of the same rules in the browser, using the same sample data. It does not write to Oracle. The genuine version runs on my laptop.*" Then show the SQL files in VS Code (section 8) as evidence of the database work.

4. Login

The login page has two tabs, **Customer** and **Fraud Analyst**, and a **Use demo account** button that fills in the email for the selected tab. The source contains no passwords.

Role	Email	Linked to
Fraud analyst	`admin@smartbank.com`	Analyst console
Customer	`user@smartbank.com`	Demo customer #1 (displayed as *Ashmita Mazumdar*, home city Mumbai)

On your laptop today (✅ verified): the database has the older REST module, so **the password is not checked**. Type any text, for example `demo`. After sign-in an amber notice says "*Local Oracle API without server-side sign-in detected… the password is not checked.*" Data and transfers are still real Oracle.

What to say: "*For this demo the login is a demonstration mechanism, not production authentication. The project also includes a server-side login layer (`15_api_security.sql`: hashed passwords and session tokens checked by Oracle), but it is not installed on this laptop.*"

> **Do not run `scripts\register-endpoints.ps1` before the presentation.** It would install that server-side login. Login would then need the passwords you choose in it, and the `start-backend.ps1` health check (which calls `/kpis` without a session) would no longer pass. The current setup is the one that was verified.

Public demo mode: the password is not needed; click **Use demo account**, then **Sign in**.

---

5. Full demonstration walkthrough

Follow this order. The format for each feature is: **Open → Do → You see → Say → Concept.**

5.1 Analyst Overview — sign in as Fraud Analyst

- **Open:** login → **Fraud Analyst** tab → **Use demo account** → type any password → **Enter Analyst Console**.
- **You see:** *Connected to Oracle*. KPI cards: **Customers 15**, **Accounts 20**, **Transactions 102**, **Fraud Alerts 76**, **Open Alerts 73**, **High Risk Alerts 4** (✅ values on 14 Oct; they grow when you add transactions). Below them: the daily volume chart, the risk-distribution chart and a review queue.
- **Say:** "*Every number here is a live SQL count from Oracle through ORDS. The daily chart comes from the view `V_DAILY_TRANSACTION_ANALYTICS`.*"
- **Concept:** aggregate queries, views, a REST API over the database.

5.2 Transactions — search and filter

- **Open:** sidebar **Transactions** (*Transaction Intelligence*).
- **Do:** type `Lagos` in the search box, then click the **FLAGGED** filter. Try **UNDER REVIEW** too.
- **You see:** summary cards (*Total transactions, Total value, Under fraud review, Flagged (high risk)*) and a table with a separate **Payment status** column and **Fraud review** column.
- **Say:** "*A MEDIUM-risk transaction gets an alert but stays COMPLETED. Only HIGH risk changes the status to FLAGGED. That is the rule in `DETECT_FRAUD`.*"
- **Concept:** joins across `BANK_TRANSACTION`, `ACCOUNT`, `CUSTOMER`, `MERCHANT` and `DEVICE` (foreign keys).

5.3 Fraud Alerts — details and analyst decision

- **Open:** sidebar **Fraud Alerts** (*Fraud Alert Center*). Filters: **ALL / HIGH / MEDIUM** and a status drop-down.
- **Do:** click the top alert to open its details.
- **You see:** the risk score, level and reasons (for example *High transaction amount; Unrecognized device; Unusual transaction location*), plus the customer, merchant, device and assigned analyst. Buttons: **Start investigation**, **Mark false positive**, **Close alert**, **Confirm fraud**.
- **Do (writes one status change, so only on the alert you create in section 6):** click **Start investigation** or **Confirm fraud**.
- **Say:** "*The status is updated in `FRAUD_ALERT`, and the CHECK constraint `CHK_ALERT_STATUS` only allows the five valid statuses.*"
- **Concept:** UPDATE with constraints, workflow states.

5.4 Analyst Workload

- **Open:** sidebar **Analyst Workload**. ✅ Today: **Kavita Rao 37 open**, **Suresh Menon 36 open**.
- **Say:** "`GENERATE_ALERT` *gives each new alert to the analyst with the fewest OPEN alerts, so the next alert should go to Suresh Menon.*" (This is true only while those counts hold.)
- **Concept:** subquery with GROUP BY inside a procedure.

5.5 System & Settings

- **Open:** sidebar **System & Settings** → **Test all endpoints** (10 healthy, `/health` fails as expected). Scroll to see the risk model, the database object list and *Demonstration limits*.
- **Say:** "*This page tests each REST endpoint live and shows the HTTP status and response time.*"

5.6 Customer portal — sign out, then sign in as Customer

- **Open: Sign out** (top right) → **Customer** tab → **Use demo account** → any password → **Sign in to SmartBank**.
- **Dashboard:** the name **Ashmita Mazumdar** is read from the `CUSTOMER` table, and only this customer's data is shown (✅ 12 transactions today).
- **My Accounts: AC1000001** (savings, ₹45,000) and **AC1000011** (current, ₹3,00,000).
- **Transactions:** the same filters, but only this customer's rows.
- **Say:** "*Customers and accounts use a supertype/subtype design:* `CUSTOMER` → `INDIVIDUAL_CUSTOMER` / `BUSINESS_CUSTOMER`, *and* `ACCOUNT` → `SAVINGS_ACCOUNT` / `CURRENT_ACCOUNT`."
- **Note:** there is no separate *Customers* page in the analyst console. Customers are shown in the transaction and alert details and in the customer portal.

5.7 Database proof (SQL)

Show this in the **SQL** terminal (section 8). It proves the website data really is in Oracle.

---

6. Live transaction demonstration (real Oracle)

The **New Transfer** page (customer portal) calls `POST /transactions`. The ORDS handler inserts **one row** into `BANK_TRANSACTION`. Then `TRG_VALIDATE_TRANSACTION` checks it, and `TRG_FRAUD_MONITOR` runs `DETECT_FRAUD` once. The page shows the new transaction ID, the status and any alert. **Balances are not changed** (demonstration scope).

> **These steps add permanent rows to your database.** Do them yourself, only when you decide to. Each transfer adds one `BANK_TRANSACTION` row and, if the score is 40 or more, one `FRAUD_ALERT` row. Nothing else is modified, and the data is synthetic. Do **not** use any real account or real card details.

Before submitting, the form checks: amount above ₹0, at most ₹1,00,00,000 and at most 2 decimals, and that an account, payee, location and device are selected. The risk preview box lists the expected signals; it is only a hint, and the database decides.

Transfer A — expected LOW (no alert)

Field	Value
From account	`AC1000001`
Payee	`Amazon India`
Amount (₹)	`2500`
Location	`Mumbai`
Device	`DEV-AND-0001 · used on this account`

Click **Submit transfer**. **Expected:** *Transaction #… recorded*, status **COMPLETED**, *no fraud alert*. The page reports the LOW score as *below 40 (not reported by this API version)*, because the older module only returns a score when an alert

exists. Why LOW: small amount (0), known device (0), home city (0), no other transfer in the last 10 minutes (0).

Transfer B — expected HIGH (alert + FLAGGED)

Click **New transfer**, then:

Field	Value
From account	`AC1000001`
Payee	`QuickCash Loans`
Amount (₹)	`150000`
Location	`Lagos`
Device	`DEV-WIN-0003 · new for this account`

Click **Submit transfer**. **Expected:** status **FLAGGED** and *Fraud alert #… (OPEN)* with the reasons. **Score:** 40 (amount) + 20 (new device) + 20 (Lagos) = **80**, plus **15** if Transfer A was less than 10 minutes earlier, giving **95**. Both are HIGH.

Results depend on the rules and the current data. Not every transfer creates an alert.

- A device counts as *new* only until it has been used once on that account. If you rehearse tonight, tomorrow pick **another** device marked *new for this account* (for AC1000001 today: DEV-WIN-0003, DEV-MAC-0006, DEV-AND-0008, DEV-WIN-0009, DEV-IOS-0010, DEV-AND-0011).
- A third transfer within 10 minutes gets +25 for velocity and may jump a level.

Show the result

1. Customer **Transactions**: the new rows are at the top. Press **F5**: they are still there.
2. **Sign out** → analyst → **Fraud Alerts**: the new alert is at the top, assigned to the least-loaded analyst. Open it → **Start investigation**.
3. **SQL** terminal (section 8): the last 3 rows of `BANK_TRANSACTION` and `FRAUD_ALERT`, and `CALCULATE_RISK` recomputing the stored score.

Public demo mode: the same steps work there, but the button says **Simulate transfer** and the result says ***SIMULATED, not processed by Oracle***. Never describe it as an Oracle transaction.

---

7. Presentation script (short version)

The full script, with click cues, is in `docs/SMARTBANK_PRESENTATION_SCRIPT.md` (about 6 minutes). Its outline:

1. Introduction and problem (40 s) → 2. Objective and schema (60 s) → 3. Why Oracle and PL/SQL (30 s) → 4. Analyst console (60 s) → 5. Live transactions A and B (90 s) → 6. Alert and analyst workflow (40 s) → 7. SQL proof (40 s) → 8. Testing, limitations and conclusion (40 s).

---

8. Database and PL/SQL reference

All objects below exist and are **VALID** in your database (✅ checked 14 Oct 2026; `PKG_SMARTBANK_API` is *not* installed on the laptop).

Object	Purpose in SmartBank	What to show
`CALCULATE_RISK` (function)	Adds the points for amount, velocity, device and location; max 100	`database/05_functions.sql` ; query Q3
`CLASSIFY_RISK` (function)	0–39 LOW, 40–69 MEDIUM, 70–100 HIGH	Query Q4

Object	Purpose in SmartBank	What to show
`DETECT_FRAUD` (procedure)	Scores, classifies, builds the reason text, calls `GENERATE_ALERT`, sets FLAGGED for HIGH	`database/06_procedures.sql`
`GENERATE_ALERT` (procedure)	Inserts `FRAUD_ALERT` for the least-loaded analyst	Analyst Workload page
`TRG_VALIDATE_TRANSACTION` (BEFORE INSERT, row)	Rejects non-ACTIVE accounts and amounts > 1 crore	`database/07_triggers.sql`
`TRG_FRAUD_MONITOR` (compound trigger)	Collects new IDs per row, then calls `DETECT_FRAUD` AFTER STATEMENT, which avoids the mutating-table error ORA-04091	`07_triggers.sql`
`REVIEW_HIGH_RISK_ALERTS` (procedure, explicit cursor)	Moves OPEN HIGH alerts to INVESTIGATING with `FOR UPDATE` / `WHERE CURRENT OF`	`database/08_cursor.sql` (show the code; running it changes data)
`PKG_FRAUD_DETECTION` (package)	Groups the engine: `get_risk_score`, `get_risk_level`, `run_fraud_check`, `review_high_risk`	Query Q4
4 views	`V_DAILY_TRANSACTION_ANALYTICS`, `V_FRAUD_ALERT_SUMMARY`, `V_HIGH_RISK_TRANSACTIONS`, `V_CUSTOMER_TRANSACTION_SUMMARY`	Query Q5
6 indexes	`IDX_TXN_ACCOUNT`, `IDX_TXN_MERCHANT`, `IDX_TXN_DEVICE`, `IDX_TXN_TIME`, `IDX_ALERT_RISK_LEVEL`, `IDX_ALERT_STATUS`	`database/11_indexes.sql`, `database/tests/day5_explain_plan_test.sql`

Concepts in simple words

- **Primary key:** a unique ID per row (12 tables, identity columns such as `TRANSACTION_ID`).
- **Foreign key:** a link that must point to an existing row, for example a transaction's `ACCOUNT_ID` (11 foreign keys).
- **CHECK constraint:** a rule on values (9 constraints, for example `CHK_TXN_AMOUNT` amount > 0, and `CHK_ALERT_SCORE` 0–100). There are also 6 UNIQUE constraints (emails, account numbers).
- **Relationships and EER:** customers → accounts → transactions → at most one alert; supertype/subtype for customers and accounts.
- **Normalisation:** each fact is stored once (merchant and device details live in their own tables and are joined in), which avoids duplication and update anomalies.
- **Index:** a fast lookup structure for frequently searched columns.
- **View:** a saved query used like a table, for reports.
- **Trigger:** code that runs automatically on INSERT. **Procedure / function:** stored, reusable PL/SQL.
- **Transaction:** the insert, scoring and alert either all succeed or all roll back.

SQL proof (read-only; ✅ every query was run on 14 Oct 2026). In the **SQL** terminal: `sqlplus smartbank_user@localhost:1521/XEPDB1` and type **your own SMARTBANK_USER password** when asked (⚠️ only you know it).

```
-- Q1 objects
SELECT object_type, COUNT(*) FROM user_objects GROUP BY object_type ORDER BY 1;
-- Q2 newest transactions and alerts
SELECT transaction_id, amount, location, status FROM bank_transaction
 ORDER BY transaction_id DESC FETCH FIRST 3 ROWS ONLY;
SELECT alert_id, transaction_id, risk_score, risk_level, status, analyst_id
  FROM fraud_alert ORDER BY alert_id DESC FETCH FIRST 3 ROWS ONLY;
-- Q3 recompute the newest alert's score with the function (today: 80 = 80)
SELECT fa.transaction_id, fa.risk_score, calculate_risk(fa.transaction_id) AS recomputed
  FROM fraud_alert fa WHERE fa.alert_id = (SELECT MAX(alert_id) FROM fraud_alert);
-- Q4 classification and package
SELECT classify_risk(39), classify_risk(40), classify_risk(70),
       pkg_fraud_detection.get_risk_level(85) FROM dual;
-- Q5 views and constraints
SELECT * FROM v_fraud_alert_summary;
SELECT constraint_name, search_condition_vc FROM user_constraints WHERE constraint_name LIKE 'CHK%';
```

Type `exit` to leave SQL*Plus. **Do not run `13_testing.sql` during the demo:** tests 1–9 insert permanent test rows.

9. Troubleshooting

Problem	What to do (nothing here deletes data)
Oracle database unavailable	Run `start-backend.ps1` again and accept UAC. Or `services.msc` → **OracleServiceXE** → Start. Wait 1 minute.
Listener unavailable (`ORA-12541` , `ORA-12514`)	`services.msc` → **OracleOraDB21Home1TNSListener** → Start; wait about 60 s for `xepdb1` to register (`lsnrctl status`). Starting `tnslsnr` as a normal user does not work (ORA-12518).
ORDS not starting	Read the ORDS window. If the script says the pool *has no database password*: `powershell -ExecutionPolicy Bypass -File scripts\repair-ords-pool.ps1` (hidden prompt; use the existing password, **not** `-ResetPassword`).
Port 8080 already in use	The script assumes ORDS is already running. Check `http://localhost:8080/ords/smartbank/sb/api/kpis` . If another program owns 8080: `Get-NetTCPConnection -LocalPort 8080` → close that program.
Frontend not starting	In `frontend` : `npm ci` once, then `npm run dev` .
Port 5173 in use	Usually an old `npm run dev` is still running: press **Ctrl+C** in that terminal and start again. Otherwise Vite picks the next port (for example 5174) and prints it; open that URL. The laptop's current API accepts it (✅ CORS checked for 5173 and 5174).
API offline / red badge	ORDS window closed or the database is down. Run `start-backend.ps1` , then click the status card in the sidebar to re-check.
CORS error in the browser console	Usually ORDS is down: an error page carries no CORS header. Fix *API offline* first. Open the site as `http://localhost:…` , not a LAN IP.
Login problems	Pick the right tab, click **Use demo account**, and type any password (the older module doesn't check it). If it says *endpoint is not published*, the API is unreachable: see *API offline*.
No fraud alert appeared	The score was below 40, which is correct behaviour. Check the device (*new for this account*?), the location (not Mumbai?) and the amount.
Browser shows DEMO MODE	You are on the public GitHub Pages link. For the real demo use `http://localhost:5173` .

Problem	What to do (nothing here deletes data)
Missing sample data	Re-check the KPIs (`/kpis` should show 15 customers). Do **not** re-run the create or seed scripts on the existing schema: they are for an empty schema only. Use the public demo as the fallback.

---

10. Final checklist

Night before

- ☐ Laptop charged; charger packed. Windows updates paused.
- ☐ Run the startup checklist (section 3) once. ⚠️ Confirm it works.
- ☐ Optional rehearsal of Transfers A and B (this adds rows; afterwards choose another *new* device for tomorrow).
- ☐ Open the public link once on your phone and check that it shows **DEMO MODE**.
- ☐ Do **not** run `register-endpoints.ps1`, `13_testing.sql` or any create/seed script.

Morning startup

- ☐ VS Code → `C:\Projects\SmartBank`; terminals **Backend**, **Frontend**, **SQL**.
- ☐ Backend: `powershell -ExecutionPolicy Bypass -File scripts\start-backend.ps1` → *API is up*.
- ☐ Frontend: `cd frontend` → `npm run dev` → Chrome `http://localhost:5173`.

Five minutes before

- ☐ **LIVE** badge is green; **Test all endpoints** shows 10 healthy (only `/health` fails).
- ☐ Note today's KPI numbers. Know which device is still *new for this account*.
- ☐ Close unrelated tabs; zoom Chrome to 110 % so the screen is easy to read.

Demonstration order: Overview → Transactions → Fraud Alerts → Analyst Workload → (sign out) Customer dashboard → My Accounts → New Transfer A → New Transfer B → Transactions + F5 → (analyst) new alert → Start investigation → SQL proof → System & Settings → conclusion.

If the real Oracle backend fails: spend at most 5 minutes on section 9. Then switch to the public link, **say that it is simulated**, and show the SQL and PL/SQL files in VS Code plus `docs/PROJECT_STATUS.md` (recorded test results).

Never claim:

- that the public link uses Oracle, or that simulated transfers were written to the database;
- that the laptop login is secure authentication;
- that the risk score is a probability or machine learning;
- that transfers move money (balances are not changed);
- that the server-side login layer is running on the laptop (it is not installed there).

What was verified for this guide (14 Oct 2026)

- ✅ Both Oracle services running (Automatic); local API answering with live data; all fraud-engine objects VALID; every SQL query in section 8 executed read-only.
- ✅ The local website against the laptop's Oracle, read-only: sign-in for both roles, live data, transfer form (8 payees, not submitted), alerts, refresh. **12/12 checks**, no write requests.
- ✅ Public demo mode: **24/24** browser checks on the live GitHub Pages site.
- ✅ The full insert → trigger → alert path (LOW 0, HIGH 95, FLAGGED, one alert) on a disposable Oracle 23ai copy of the same scripts: 62/62 API checks.
- ⚠️ **Not run on your laptop database:** submitting a transfer, and changing an alert status. These write permanent rows, so you perform them yourself.